Project Name: Rice Detection

Creator: Kunanon Sriborthong 2211310558

Tanapat Samudinkeaw 2211311929

Pheraphon Prueksjamas 2211312141

1. หลักการทำงานของระบบตรวจสอบและคัดแยกคุณภาพเมล็ดข้าว

ระบบถูกออกแบบโดยยึดหลักการของระบบตรวจสอบอัตโนมัติ (Automated Inspection System) ซึ่งประกอบด้วยขั้นตอนเชิงตรรกะดังนี้:

- **การปรับปรุงคุณภาพภาพ (Image Enhancement):** ระบบใช้ **Bilateral Filter** เพื่อลดสัญญาณรบกวน (Noise) โดยที่ยังคงความคมชัดของขอบวัตถุไว้ ซึ่งต่างจาก Gaussian Blur ทั่วไปที่มักทำให้ขอบเมล็ดข้าวเบลอ
- **การแยกวัตถุโดยใช้พื้นหลังอ้างอิง (Background Subtraction via ROI):** เพื่อความแม่นยำ ระบบกำหนดให้ใช้ถาดสีแดงเป็นพื้นหลัง และสร้าง **Masking** ผ่านปริภูมิสี HSV เพื่อตัดพื้นที่ที่ไม่เกี่ยวข้องออกก่อนเริ่มวิเคราะห์เมล็ดข้าว
- **กระบวนการแบ่งส่วนภาพ (Image Segmentation):** เป็นการเปลี่ยนภาพจากระดับสีเทา (Grayscale) ให้เป็นภาพขาวดำ (Binary Image) โดยใช้ **Otsu's Thresholding** ซึ่งเป็นอัลกอริทึมที่คำนวณค่าขีดแบ่งพิกเซลที่เหมาะสมที่สุดจากฮิสโตแกรมของภาพโดยอัตโนมัติ
- **การวิเคราะห์ทางสัณฐานวิทยา (Morphological Operations):** ใช้การทำ **Opening** (การกัดกร่อนแล้วขยาย) เพื่อกำจัดจุดรบกวนขนาดเล็กและช่วยแยกเมล็ดข้าวที่วางแตะกันให้ขาดออกจากกัน
- **การสกัดคุณลักษณะและการจำแนกประเภท (Feature Extraction & Classification):** ระบบวิเคราะห์คุณลักษณะทางกายภาพ เช่น พื้นที่ (Area), ความทึบ (Solidity) และอัตราส่วน (Aspect Ratio) เพื่อจำแนกเมล็ดข้าวตามเกณฑ์ที่กำหนด

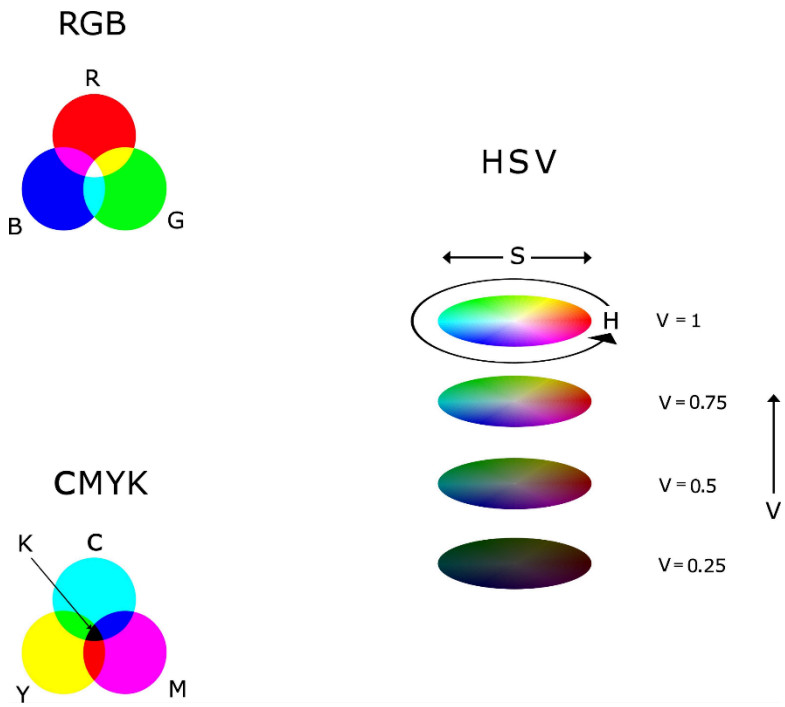
2. หลักการทางคณิตศาสตร์เบื้องหลังระบบ (Mathematical Foundations)

ในprojectนี้มีการทำงาน 2 ส่วนหลัก คือ แบบ realtime ที่ทำงานอัติโนมัติโดยตรวจจับเมล็ดข้าวผ่านกล้องของผู้ใช้กับการตรวจจับเมล็ดข้าวจากภาพนิ่งที่ผู้ใช้อัปโหลดเข้ามาเอง โดยการประมวลผลนี้อาศัยโมเดลทางคณิตศาสตร์ที่สำคัญ 4 ส่วนหลัก:

2.1 ปริภูมิสี HSV (Color Space Transformation)(ทั้ง2ระบบ)

ในระบบดิจิทัลปกติ ภาพจะถูกเก็บในรูปแบบ RGB (Red, Green, Blue) แต่ระบบนี้จะแปลงเป็น HSV (Hue, Saturation, Value) เนื่องจากการคำนวณทางคณิตศาสตร์เพื่อแยกสีใน RGB ทำได้ยากเมื่อความสว่างเปลี่ยนไป

- **Hue** (H)**:** มุมของสีบนวงล้อสี ($0^0 - 360^0$) ใน OpenCV จะถูก Map ให้อยู่ในช่วง 0-180
- **Saturation** (S)**:** ความอิ่มตัวของสี ($0 - 255$)
- **Value** (V)**:** ค่าความสว่าง ($0 - 255$)



2.2 อัลกอริทึม Otsu's Thresholding(เฉพาะภาพนิ่ง)

คณิตศาสตร์ที่ใช้คือการหาค่าขีดเริ่ม T ที่ทำให้ Within-class Variance (σ_w^2) มีค่าน้อยที่สุด:

$$\sigma_w^2(T) = w_0(T)\sigma_0^2(T) + w_1(T)\sigma_1^2(T)$$

โดยที่ w คือค่าน้ำหนักของกลุ่มพิกเซลพื้นหลังและวัตถุ และ σ^2 คือค่าความแปรปรวนของแต่ละกลุ่มพิกเซล

2.3 การแยกเมล็ดข้าวด้วย Distance Transform(real time)

ในกรณีเมล็ดข้าวเบียดกัน ระบบใช้ Euclidean Distance คำนวณระยะห่างจากพิกเซลวัตถุไปยังพิกเซลพื้นหลังที่ใกล้ที่สุด:

$$d(p,\ q) \ = \ \sqrt{\left(p_x - q_x\right)^2 + \left(p_y - q_y\right)^2}$$

พิกเซลที่มีค่า d สูงที่สุดคือ "จุดศูนย์กลางเมล็ด" ซึ่งระบบจะใช้เป็นจุดเริ่มต้นในการนับจำนวนเมล็ดแยกกัน

2.4 การคำนวณคุณลักษณะเชิงรูปทรง (Shape Features)

(ทั้ง2ระบบ)

ระบบใช้หลักเรขาคณิตวิเคราะห์ Contours ของเมล็ดข้าว:

Solidity (ความทึบของวัตถุ): ใช้แยกเมล็ดข้าวที่แตกหักหรือเว้าแหว่ง

$$solidity \ = \ \frac{Area\ of\ Contour}{Area\ of\ Convex\ Hull}$$

Aspect Ratio (อัตราส่วนภาพ): ใช้จำแนกพันธุ์ข้าวหรือตัดสิ่งเจือปนที่เป็นเส้นยาว

$$Aspect\ Ratio = \frac{Width\ of\ Bounding\ Box}{Height\ of\ Bounding\ Box}$$

- **Fail Ratio (อัตราส่วนเมล็ดเสีย):** คำนวณจากสัดส่วนพิกเซลที่ตรงกับสีเสีย p_{fail} ต่อพิกเซลทั้งหมดของเมล็ด p_{total} :

$$Ratio = \frac{p_{fail}}{p_{total}}$$

หาก $Ratio > yellow_sensitivity$ ระบบจะตัดสินว่าเป็น **Fail**

2.5การคำนวณสัดส่วนเพื่อปรับขนาดภาพ (Image Resizing)

(เฉพาะภาพนิ่ง)

ในโค้ดมีการล็อคความกว้างของภาพไว้ที่ 800 พิกเซล โดยใช้วิธีคำนวณความสูงใหม่เพื่อรักษาอัตราส่วน (Aspect Ratio) ของภาพต้นฉบับไว้ ดังสมการ:

$$H_{new} = H_{old}\ x\ (\frac{W_{new}}{W_{old}})$$

3. โครงสร้างการทำงานของโปรแกรม (Program Architecture)

การทำงานของโปรแกรมจะแบ่งออกเป็น 2 ส่วนหลัก คือส่วน Interface (Streamlit) ที่ทำหน้าที่รับค่าจากผู้ใช้ และส่วน Processing (OpenCV) ที่ทำหน้าที่คำนวณทางคณิตศาสตร์ โดยมีขั้นตอนดังนี้:

3.1 การเตรียมภาพและปรับสภาพ (Image Pre-processing)

เมื่อโปรแกรมได้รับภาพ (ไม่ว่าจะจากการอัปโหลดหรือกล้องสด):

- **Bilateral Filter:** โปรแกรมจะใช้ตัวกรองนี้เพื่อลดสัญญาณรบกวน (Noise) จุดเด่นคือการทำให้พื้นผิวของเมล็ดข้าวเนียนขึ้น แต่ยังคงรักษาความคมชัดของขอบเมล็ดไว้ (Edge-Preserving) ซึ่งจำเป็นมากต่อการวัดขนาด
- **HSV Conversion:** แปลงภาพจากระบบสี BGR เป็น HSV เพื่อแยกค่าสีออกจากความสว่าง ช่วยให้การตรวจจับเมล็ดข้าวสีขาวหรือสีเหลืองเสื่อมคุณภาพมีความแม่นยำสูง แม้แสงในห้องจะเปลี่ยนไป

3.2 การแยกส่วนภาพ (Image Segmentation)

โปรแกรมจะทำการแยกเมล็ดข้าวออกจากพื้นหลังด้วย 2 กลไกขนานกัน:

- **Color Thresholding:** สร้างหน้ากาก (Mask) จากช่วงสีที่ผู้ใช้กำหนดผ่าน Slider (H-S-V) เพื่อระบุพิกเซลที่น่าจะเป็น "เมล็ดข้าว" หรือ "เมล็ดเสีย"
- **Otsu's Binarization:** ใช้คณิตศาสตร์เชิงสถิติวิเคราะห์ Histogram ของภาพเพื่อหาค่า "จุดตัดความเข้มแสง" (Threshold) ที่เหมาะสมที่สุดในการแยกวัตถุสีขาวออกจากพื้นหลังโดยอัตโนมัติ

3.3 การแยกเมล็ดที่วางซ้อนกัน (Distance Transform & Watershed)

ในกรณีที่เมล็ดข้าววางเบียดกันจนกลายเป็นก้อนเดียว โปรแกรมใช้เทคนิคทางคณิตศาสตร์ขั้นสูง:

- **Distance Transform:** คำนวณระยะห่างของทุกพิกเซลขาวไปยังพิกเซลดำที่ใกล้ที่สุด พิกเซลที่อยู่ "ใจกลาง" เมล็ดจะมีค่าสูงสุด
- **Peak Detection:** ใช้จุดที่มีค่าระยะทางสูงสุดเป็นจุดศูนย์กลาง (Seed) เพื่อระบุว่าตรงนี้คือ 1 เมล็ด แม้ขอบจะติดกับเมล็ดอื่นก็ตาม

3.4 การวิเคราะห์คุณลักษณะ (Feature Extraction & Logic)

โปรแกรมจะวนลูปตรวจสอบวัตถุ (Contours) ทีละชิ้น โดยใช้เกณฑ์ตัดสินดังนี้:

- **Area Filtering:** หากพื้นที่พิกเซลน้อยเกินไปจะถูกนับเป็น "ฝุ่น" หากมากเกินไปจะถูกนับเป็น "สิ่งเจือปนขนาดใหญ่"
- **Solidity Calculation:** คำนวณความแน่นของรูปทรง หากเมล็ดข้าวมีค่านี้ต่ำ หมายถึงเมล็ดหักหรือมีรอยแหว่ง
- **Color Ratio Analysis:** โปรแกรมจะนับพิกเซลสีเสีย (เช่น สีเหลืองเข้ม/ดำ) ที่อยู่ภายในขอบเขตของเมล็ดนั้นๆ
    - *สูตร:* $Fail\ Ratio = \frac{Fail\ Pixels}{Total\ Pixels\ of\ Grain}$
    - หากค่า $Ratio$ สูงกว่าที่ผู้ใช้ตั้งไว้ $(Yellow\ Sensitivity)$ โปรแกรมจะตีตราว่าเป็น **"Fail" (เมล็ดเสีย)**

3.5 การแสดงผลลัพธ์ (Output Visualization)

- **Bounding Box & Label:** โปรแกรมจะวาดกรอบสี่เหลี่ยมล้อมรอบเมล็ดข้าว พร้อมใส่สี (เขียว = Pass / แดง = Fail) และระบุ ID ของเมล็ดนั้นๆ
- **Real-time Dashboard:** ข้อมูลเชิงสถิติ (จำนวนรวม, จำนวนผ่าน, จำนวนเสีย) จะถูกส่งกลับไปแสดงที่หน้าจอ Streamlit ในรูปแบบ Metric ที่อ่านง่าย

4. รูปแบบการทำงาน

4.1 การทำงานในโหมดภาพนิ่ง (Static Image Processing)

ในโหมดนี้โปรแกรมจะเน้นความแม่นยำสูงและการควบคุมสภาพแวดล้อม (ถาดวาง) เพื่อให้ได้ผลลัพธ์ที่ดีที่สุดจากไฟล์ที่อัปโหลด

- **การเตรียมพื้นที่สนใจ (Tray Segmentation):** โปรแกรมจะค้นหา "ถาดสีแดง" โดยใช้ช่วงสี HSV ของสีแดง 2 ช่วง (lower_red1-2) เพื่อสร้าง Mask และตัดพื้นหลังที่ไม่ใช่อุปกรณ์การทดลองออก
- **การรวม Mask (Hybrid Thresholding):** ใช้การรวมกันระหว่าง mask_white (จากช่วงสีขาวใน HSV) และ thresh_gray (จาก Otsu Threshold) เพื่อระบุตำแหน่งเมล็ดข้าวให้แม่นยำที่สุด
- **การคัดกรองรูปร่าง (Shape Filtering):** * **ขนาด (Area):** กรองวัตถุที่มีขนาดระหว่าง 300 ถึง 4000 พิกเซล
    - **ความทึบ (Solidity):** ต้องมีค่ามากกว่า 0.75 เพื่อคัดเมล็ดข้าวที่หักหรือแหว่งมากๆ ออก
    - **สัดส่วน (Aspect Ratio):** ต้องอยู่ระหว่าง 0.25 - 4.0 เพื่อป้องกันการตรวจจับวัตถุแปลกปลอมที่เป็นเส้นยาว
- **ผลลัพธ์:** แสดงจำนวนเมล็ดข้าวที่ "ผ่านเกณฑ์ (Good)" เพียงอย่างเดียว

4.2 การทำงานในโหมดเรียลไทม์ (Real-time Video Processing)

โหมดนี้ถูกออกแบบมาเพื่อเน้นความเร็วในการประมวลผลควบคู่ไปกับความแม่นยำ โดยมีการใช้ตรรกะระดับสูงเพื่อจัดการกับข้อจำกัดทางกายภาพ เช่น เมล็ดข้าวที่วางเบียดกันหรือสัญญาณรบกวนจากสภาวะแสง ดังนี้:

1. การลดสัญญาณรบกวนแบบพิเศษ (Advanced Noise Reduction)

- **เทคนิคที่ใช้:** เลือกใช้ **Bilateral Filter** แทนการใช้ Gaussian Blur แบบปกติ
- **เหตุผล:** Bilateral Filter สามารถลด Noise หรือ "Grain" จากเซนเซอร์กล้องได้ดีเยี่ยม โดยยังคงรักษา **ความคมชัดของขอบเมล็ดข้าว (Edge Preserving)** ไว้อย่างครบถ้วน ซึ่งสำคัญมากต่อการคำนวณพื้นที่ (Area) และความยาว (Length) ของเมล็ดข้าวในขั้นตอนถัดไป

2. การแยกเมล็ดที่ติดกัน (Grain Segmentation)

ในกรณีที่ข้าวถูกเทวางจนเบียดกัน โปรแกรมจะใช้กระบวนการทางคณิตศาสตร์เข้ามาช่วย:

- **Distance Transform:** คำนวณระยะทางจากพิกเซลสีขาวไปยังขอบพิกเซลสีดำที่ใกล้ที่สุดเพื่อหาจุดลึกที่สุดของเมล็ด
- **Sure Foreground:** ใช้จุดยอดสูงสุด (Peak) ของค่าระยะทางเพื่อระบุ "ตำแหน่งแกนกลางเมล็ด" (Seed Points) ทำให้โปรแกรมสามารถแยก Contours ของเมล็ดที่วางชิดกันออกจากกันได้ ไม่มองเป็นก้อนเดียว

3. การวิเคราะห์คุณภาพรายเมล็ด (Individual Grain Analysis)

เมื่อแยกเมล็ดได้แล้ว โปรแกรมจะวนลูปตรวจสอบคุณสมบัติในระดับพิกเซลของทุกเมล็ด:

- **การวิเคราะห์สีเสีย (Color Analysis):** คำนวณหาค่า `fail_ratio` หรือสัดส่วนพิกเซลที่ตรงกับช่วงสีที่ตั้งไว้ (HSV Range) ต่อพื้นที่ทั้งหมดของเมล็ดนั้น โดยทำงานร่วมกับค่าความไวที่ผู้ใช้กำหนด (Sensitivity)
- **การวิเคราะห์รูปทรง (Morphological Analysis):** ใช้ `cv2.minAreaRect` เพื่อหาความยาวเมล็ดที่หมุนตามองศาจริง และตรวจสอบค่า **Solidity** เพื่อแยกเมล็ดที่สมบูรณ์ออกจากเมล็ดหักหรือสิ่งเจือปน

4. ผลลัพธ์และการตอบสนอง (Real-time Output)

- **Visualization:** แสดงสถานะบนหน้าจอวิดีโอทันที โดยแยกสีอย่างชัดเจน:
    - **สีเขียว (Pass):** เมล็ดที่ผ่านเกณฑ์ทั้งด้านสี, ความยาว และรูปทรง
    - **สีแดง (Fail):** เมล็ดที่ไม่ผ่านเกณฑ์ พร้อมระบุสาเหตุประกอบ (เช่น Color, Shape, หรือ Short/Broken)
- **Statistics:** สรุปยอดรวมจำนวนเมล็ดที่ผ่านและไม่ผ่านเกณฑ์แบบ Real-time เพื่อนำไปวิเคราะห์สถิติต่อไป

Static Image Vs Realtime Image

หัวข้อเปรียบเทียบ	โหมดภาพนิ่ง (rice_logic.py)	โหมดเรียลไทม์ (ricerealtime_logic.py)
เป้าหมายหลัก	คัดกรองรูปร่างเมล็ดข้าวที่สมบูรณ์	คัดแยกเมล็ดดี/เมล็ดเสีย และนับจำนวน
การจัดการพื้นหลัง	ค้นหาถาดสีแดงเพื่อกำหนด ROI	ใช้การปรับค่า HSV จากผู้ใช้โดยตรง
การแยกเมล็ดติดกัน	ใช้ Morphological Opening	ใช้ Distance Transform (แม่นยำกว่า)
เกณฑ์การตัดสิน	พื้นที่, ความทึบ, อัตราส่วนภาพ	อัตราส่วนสีเสีย (Fail Ratio)
การกรอง Noise	Gaussian Blur	Bilateral Filter (รักษาขอบภาพ)

5. ส่วนการทำงานของโค้ด

เราแบ่งการทำงานออกเป็น 2 ส่วนหลัก คือ rice_logic.py สำหรับประมวลผลจากภาพนิ่ง (Image) และ ricerealtime_logic.py สำหรับประมวลผลวิดีโอสด (Real-time)

5.1 rice_logic.py

```
import cv2
import numpy as np

# --- พารามิเตอร์ช่วงสีสำหรับการคัดแยก (ปรับแต่งได้) ---
# 1. ช่วงสีแดงสำหรับจับพื้นที่ถาด (HSV แยกเป็น 2 ช่วงเพราะสีแดงอยู่ตรงขอบสเกล)
lower_red1 = np.array([0, 70, 50])
upper_red1 = np.array([10, 255, 255])
lower_red2 = np.array([170, 70, 50])
upper_red2 = np.array([180, 255, 255])

# 2. ช่วงสีขาวสำหรับจับเมล็ดข้าว (ความอิ่มตัวสีต่ำ, ความสว่างสูง)
lower_white = np.array([0, 0, 150])
upper_white = np.array([180, 60, 255])

# ==========================================
# โหมด 1: สำหรับภาพนิ่ง (Upload Image)
# ==========================================
def process_rice_image(img, dist_val=0.4):
    # ปรับขนาดภาพให้กว้าง 800px เสมอ
    h, w = img.shape[:2]
    new_w = 800
```

```
    new_h = int(h * (new_w / w))
    img = cv2.resize(img, (new_w, new_h))
    original = img.copy()

    # 1. สร้างหน้ากาก (Mask) ค้นหาถาดสีแดงเพื่อตัดพื้นหลังโต๊ะออก
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    mask_red = cv2.bitwise_or(cv2.inRange(hsv, lower_red1,
upper_red1),
                              cv2.inRange(hsv, lower_red2,
upper_red2))

    kernel_open = cv2.getStructuringElement(cv2.MORPH_RECT, (15, 15))
    mask_red = cv2.morphologyEx(mask_red, cv2.MORPH_OPEN, kernel_open)

    contours_red, _ = cv2.findContours(mask_red, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    mask_tray = np.zeros(img.shape[:2], dtype="uint8")

    if contours_red:
        cnt_tray = max(contours_red, key=cv2.contourArea)
        hull_tray = cv2.convexHull(cnt_tray)
        cv2.drawContours(mask_tray, [hull_tray], -1, 255, -1)

    # เก็บเฉพาะภาพที่อยู่ข้างในพื้นที่ถาดสีแดง
```

```
    roi = cv2.bitwise_and(img, img, mask=mask_tray) if contours_red
else img


    # 2. ค้นหาเมล็ดข้าวด้วยสีขาวและ Otsu Threshold
    hsv_roi = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)
    mask_white = cv2.inRange(hsv_roi, lower_white, upper_white)

    gray_roi = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
    blur_gray = cv2.GaussianBlur(gray_roi, (7, 7), 0)
    _, thresh_gray = cv2.threshold(blur_gray, 0, 255,
cv2.THRESH_BINARY + cv2.THRESH_OTSU)

    # รวมความแม่นยำจากสีขาวและความสว่างเข้าด้วยกัน
    mask_rice = cv2.bitwise_and(thresh_gray, mask_white)

    # ลบ Noise เล็กๆ และแยกเมล็ดที่อาจจะแตะกันอยู่
    kernel_ellip = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,
5))
    mask_rice = cv2.morphologyEx(mask_rice, cv2.MORPH_OPEN,
kernel_ellip, iterations=1)

    # 3. วิเคราะห์และคัดกรองรูปร่าง
    contours_rice, _ = cv2.findContours(mask_rice, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    stats = {"Good": 0}
```

```
    for c in contours_rice:
        area = cv2.contourArea(c)
        # กรอง 1: ขนาด (อิงภาพ 800px)
        if area < 300 or area > 4000:
            continue

        # กรอง 2: ความทึบ ตัดคีมคีบและข้าวที่แหว่งมากๆ
        hull = cv2.convexHull(c)
        hull_area = cv2.contourArea(hull)
        if hull_area == 0: continue
        solidity = float(area) / hull_area
        if solidity < 0.75: continue

        # กรอง 3: สัดส่วน ตัดวัตถุเส้นยาวๆ
        x, y, w, h = cv2.boundingRect(c)
        aspect_ratio = float(w) / float(h)
        if aspect_ratio < 0.25 or aspect_ratio > 4.0: continue

        stats["Good"] += 1
        cv2.rectangle(original, (x, y), (x+w, y+h), (0, 255, 0), 2)
        cv2.putText(original, "Good", (x, y-5),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
```

```
    return original, stats
```

5.2 ricerealtime_logic.py

```
import cv2

import numpy as np


def process_rice_logic(img, h_range, s_range, v_range):

    # --- Configuration ---

    FAIL_THRESHOLD = 0.15      # สัดส่วนสีเสียในเมล็ด (15%)

    MIN_GRAIN_AREA = 200       # ขนาดพื้นที่ขั้นต่ำของเมล็ดข้าว

    SOLIDITY_THRESHOLD = 0.94  # ความ "ตัน" ของเมล็ด (ปรับให้สูงขึ้นตามคำแนะนำก่อน
หน้า)

    MIN_GRAIN_LENGTH = 50      # ความยาวขั้นต่ำของเมล็ด (ต้องปรับค่านี้ตามหน้างานจริง)


    # --- 1. Preprocessing ---

    blurred = cv2.bilateralFilter(img, 9, 75, 75)

    hsv = cv2.cvtColor(blurred, cv2.COLOR_BGR2HSV)


    lower_fail = np.array([h_range[0], s_range[0], v_range[0]])

    upper_fail = np.array([h_range[1], s_range[1], v_range[1]])

    fail_mask = cv2.inRange(hsv, lower_fail, upper_fail)


    # --- 2. Segmentation ---

    gray = cv2.cvtColor(blurred, cv2.COLOR_BGR2GRAY)

    _, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)


    kernel = np.ones((3,3), np.uint8)

    opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=1)
```

```
    # --- 3. Contour Detection ---
    contours, _ = cv2.findContours(opening, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

    display_img = img.copy()
    stats = {"Pass": 0, "Fail": 0}

    for c in contours:
        # กรองพื้นที่เล็กเกินไป (Noise)
        area = cv2.contourArea(c)
        if area < MIN_GRAIN_AREA:
            continue

        # --- กฎข้อที่ 1: ตรวจสอบรูปทรง (Solidity) ---
        hull = cv2.convexHull(c)
        hull_area = cv2.contourArea(hull)
        solidity = float(area) / hull_area if hull_area > 0 else 0

        is_solidity_fail = solidity < SOLIDITY_THRESHOLD

        # --- กฎข้อที่ 2: ตรวจสอบความยาว (minAreaRect) ---
        # ใช้ minAreaRect เพื่อหาขนาดเมล็ดที่หมุนตามองศาจริงของเม็ดข้าว
        rect = cv2.minAreaRect(c)
        # rect[1] จะได้ (width, height) เรานำมาเรียงใหม่ให้ด้านที่ยาวที่สุดเป็น length
เสมอ
        width, length = sorted(rect[1])
```

```
        is_length_fail = length < MIN_GRAIN_LENGTH # ตรวจสอบเมล็ดหัก/สั้น

        # --- กฎข้อที่ 3: ตรวจสอบสีเสีย (Color Analysis) ---
        mask_single_grain = np.zeros_like(opening)
        cv2.drawContours(mask_single_grain, [c], -1, 255, -1)

        total_pixels = cv2.countNonZero(mask_single_grain)
        grain_fail_pixels = cv2.bitwise_and(mask_single_grain, fail_mask)
        fail_pixels_count = cv2.countNonZero(grain_fail_pixels)

        fail_ratio = fail_pixels_count / (total_pixels + 1e-5)
        is_color_fail = fail_ratio > FAIL_THRESHOLD

        # --- สรุปผลการตรวจสอบ ---
        x, y, w, h = cv2.boundingRect(c)

        # ถ้านิยามของ Fail คือ หัก OR สีเสีย OR ทรงเบี้ยว
        if is_color_fail or is_solidity_fail or is_length_fail:
            # ลำดับการแสดงเหตุผล
            if is_color_fail:
                reason = "Color"
            elif is_length_fail:
                reason = "Short/Broken"
            else:
                reason = "Shape"
```

```
            label = f"Fail ({reason})"
            color = (0, 0, 255) # สีแดง
            stats["Fail"] += 1
        else:
            label = "Pass"
            color = (0, 255, 0) # สีเขียว
            stats["Pass"] += 1

        # วาดเส้นรอบรูปและแสดงสถานะ
        cv2.drawContours(display_img, [c], -1, color, 2)
        # แสดงความยาวจริง (Optional - สำหรับ Debug)
        cv2.putText(display_img, f"{label} L:{int(length)}", (x, y-7),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.4, color, 1)

    return display_img, stats
```

6. คำสั่งการใช้งาน

คำสั่ง	ความหมาย / ใช้งานเพื่อ
cv2.bilateralFilter()	ลดสัญญาณรบกวน (Noise) ในขณะที่ยังคงความคมชัดของขอบวัตถุไว้ เพื่อให้ขอบเมล็ดข้าวไม่เบลอ
cv2.cvtColor()	แปลงปริภูมิสี (Color Space) เช่น จาก BGR เป็น HSV เพื่อให้ง่ายต่อการแยกแยะเฉดสีและความสว่าง
cv2.inRange()	สร้างหน้ากาก (Mask) เพื่อดึงเฉพาะพิกเซลที่มีค่าสีอยู่ในช่วงที่กำหนด (เช่น หาพิกเซลสีเสีย หรือสีขาวของเมล็ดข้าว)
cv2.threshold()	การทำ Thresholding เพื่อแยกวัตถุออกจากพื้นหลัง โดยใช้ Otsu's method เพื่อหาค่าขีดแบ่งอัตโนมัติ
cv2.getStructuringElement()	สร้างรูปทรงโครงสร้าง (Kernel) เช่น สี่เหลี่ยมหรือวงรี เพื่อใช้ในงาน Morphological Operations
cv2.morphologyEx()	การปรับแต่งรูปทรงทางสัณฐานวิทยา เช่น การทำ Opening เพื่อกำจัดจุดรบกวนขนาดเล็กและแยกขอบที่ติดกัน
cv2.findContours()	ค้นหาเส้นรอบรูปของวัตถุที่ผ่านการทำ Segmentation แล้ว เพื่อนำไปวิเคราะห์รูปร่างต่อไป

cv2.contourArea()	คำนวณพื้นที่ (จำนวนพิกเซล) ของเส้นรอบรูปที่พบ เพื่อใช้กรองขนาดเมล็ดข้าวหรือฝุ่น
cv2.convexHull()	สร้างกรอบหลายเหลี่ยมแบบนูนที่หุ้มวัตถุ เพื่อนำไปคำนวณค่าความทึบ (Solidity)
cv2.boundingRect()	สร้างกรอบสี่เหลี่ยมแนวตั้งที่ล้อมรอบวัตถุ เพื่อหาค่าพิกัด กว้าง และสูง
cv2.distanceTransform()	คำนวณระยะห่างจากพิกเซลวัตถุไปยังพื้นหลังที่ใกล้ที่สุด เพื่อหาจุดศูนย์กลางของเมล็ดข้าวและแยกเมล็ดที่วางติดกัน
cv2.bitwise_and()	การรวมภาพหรือ Mask สองภาพเข้าด้วยกันด้วยตรรกะแบบ AND (เช่น การตัดภาพให้เหลือเฉพาะพื้นที่ถาด)
cv2.countNonZero()	นับจำนวนพิกเซลที่ไม่ใช่ศูนย์ (พิกเซลสีขาว) ใน Mask เพื่อนำมาคำนวณหาอัตราส่วนของเมล็ดเสีย
cv2.rectangle()	วาดรูปสี่เหลี่ยมลงบนภาพเพื่อระบุตำแหน่งของเมล็ดข้าวที่ตรวจพบ
cv2.putText()	เขียนข้อความลงบนภาพ เช่น ระบุสถานะ "Good", "Pass" หรือ "Fail"